1. Introduction

This document outlines what our system needs to do and how well it needs to do it. We're building an application called TransitPredict that shows real-time train and bus information but also predicts how likely services are to be cancelled or delayed based on historical patterns, weather, holidays, and industrial action.

The app will run as both a website and a desktop application using Electron.

2. Functional Requirements

2.1 Real-Time Data Display

ID	Requirement	Prior
FR-01	The system shall display live train departure/arrival times for any UK station	Must
FR-02	The system shall display live bus departure times for supported regions	Must
FR-03	Users shall be able to search for stations/stops by name or code	Must
FR-04	The system shall show platform numbers where available	Shou
FR-05	The system shall display the operator running each service	Shou
FR-06	Users shall be able to view the calling points for any service	Must
FR-07	The system shall update live data at least every 30 seconds	Must
FR-08	The system shall show whether a service is on time, delayed, or cancelled	Must

2.2 Prediction Engine

ID	Requirement	Prio
FR-09	The system shall display a cancellation probability percentage for each service	Mus
FR-10	The system shall factor in current weather conditions when calculating predictions	Mus
FR-11	The system shall factor in whether today is a public holiday	Mus
FR-12	The system shall factor in any announced strikes affecting the operator	Mus
FR-13	The system shall use at least 6 months of historical performance data	Sho
FR-14	Users shall be able to see what factors are influencing the prediction	Sho
FR-15	The prediction shall update when conditions change (weather worsening etc)	Cou

ID	Requirement	Pric
FR-16	The system shall learn from new data as it comes in	Cou

2.3 User Features

ID	Requirement	Prio
FR-17	Users shall be able to save favourite stations/stops	Shou
FR-18	Users shall be able to view recently searched locations	Shou
FR-19	The desktop app shall work offline for viewing saved favourites (without live data)	Cou
FR-20	Users shall be able to set up alerts for specific services	Cou
FR-21	The system shall remember user preferences between sessions	Shou

2.4 Data Management

ID	Requirement	Pri
FR-22	The system shall store historical service performance data	Mu
FR-23	The system shall pull weather data from an external API	Mu
FR-24	The system shall maintain a database of UK public holidays	Mu
FR-25	The system shall allow manual entry of strike information	Mu
FR-26	The system shall automatically check for strike announcements from RSS feeds	Cou

3. Non-Functional Requirements

3.1 Performance

ID	Requirement	Target
NFR-01	Page load time	Under 3 seconds on standard broadband
NFR-02	Search response time	Under 1 second
NFR-03	Prediction calculation time	Under 2 seconds per service
NFR-04	Concurrent users supported	At least 100
NFR-05	API response time	Under 500ms for cached data

3.2 Reliability

ID	Requirement	Target
NFR-06	System uptime	95% availability
NFR-07	Data freshness	Live data no more than 60 seconds old
NFR-08	Graceful degradation	System shows cached data if live feed fails

3.3 Usability

ID	Requirement	Target
NFR-09	Learnability	New user can find departure info within 30 seconds
NFR-10	Accessibility	Meet WCAG 2.1 Level AA
NFR-11	Mobile responsiveness	Web version usable on screens 320px and up
NFR-12	Desktop app platforms	Windows 10+ and macOS 11+

3.4 Security

ID	Requirement	Target
NFR-13	Data transmission	All API calls over HTTPS
NFR-14	API keys	Not exposed in client-side code
NFR-15	User data	No personally identifiable information stored without consent

4. Constraints

- Must be completable within the module timeframe (roughly 8 weeks of development)
- Team of 6 students with varying experience levels
- Budget is effectively zero except for cheap hosting/domains
- Dependent on third-party APIs remaining available and free/cheap
- Real Time Trains API has rate limits we need to work within

5. Assumptions

- The RTT API will remain accessible with a free or educational tier
- We can find a suitable bus data API (probably Traveline or similar)

- Historical performance data is available somewhere (either via API or we scrape/collect it)
- Weather APIs have free tiers with enough calls for our needs
- Users have a reasonable internet connection

6. Out of Scope

These are things we discussed but decided not to include:

- Native mobile apps (iOS/Android) - just web and Electron
- Journey planning between stations
- Ticket purchasing integration
- Full UK coverage for buses (we'll pick 2-3 regions to prove the concept)
- Real-time vehicle tracking on maps
- User accounts with authentication (might add if time permits)

7. Acceptance Criteria

The system will be considered complete when:

1. A user can search for any UK train station and see live departures
2. A user can search for bus stops in our supported regions and see live departures
3. Each service shows a cancellation/delay probability percentage
4. The prediction takes into account at least: historical data, weather, holidays
5. Both the web version and Electron desktop app are functional
6. The system handles API failures gracefully without crashing